

# pypresso

what it computes, and how to ask for it

Plane-wave DFT in Python and JAX, validated against Quantum ESPRESSO

August 30, 2026

## Abstract

User documentation for the feature set: every capability, the equation behind it, a snippet that runs it, what it was validated against, and *what it refuses*. Two conventions run through the whole document.

**Everything is validated against Quantum ESPRESSO on the same input**, and the agreement is quoted per feature rather than claimed in general. Where a number is missing it is because the comparison does not exist, and that is said rather than hidden.

**Anything not implemented is refused by name**, with an error saying what is missing — never silently approximated. The “Refuses” notes are as much of the documentation as the features: they are the promise that a run which starts is a run whose physics is there.

---

**One equation is written down and the rest are derived from it.** Almost everything below is a derivative of the total energy, taken by the compiler rather than by hand:

$$E[\{\psi\}, \tau, \varepsilon] = T_s + E_H + E_{xc} + E_{\text{loc}} + E_{\text{nl}} + E_{\text{Ewald}}$$

| quantity                                 | what it is                                                      | how it is obtained                        |
|------------------------------------------|-----------------------------------------------------------------|-------------------------------------------|
| $v_{xc}(\mathbf{r})$                     | $\delta E_{xc} / \delta n$                                      | jax.grad of the energy density            |
| $\mathbf{F}_I$                           | $-\partial E / \partial \boldsymbol{\tau}_I$                    | jax.grad at frozen $\psi$                 |
| $\sigma_{ij}$                            | $-\Omega^{-1} \partial E / \partial \varepsilon_{ij}$           | jax.grad along a strain                   |
| $C_{I\alpha, J\beta}$                    | $\partial^2 E / \partial \tau_{I\alpha} \partial \tau_{J\beta}$ | jvp of the force                          |
| $Z_{I, \alpha\beta}^*$                   | $\partial F_{I\alpha} / \partial \mathcal{E}_\beta$             | jvp of the force along the field response |
| $\partial \epsilon_{ij} / \partial \tau$ | Raman                                                           | jvp of the second-order energy            |

The “frozen  $\psi$ ” is what makes this exact rather than approximate: at self-consistency the energy is stationary with respect to the wavefunctions, so the terms from  $\partial \psi / \partial \lambda$  vanish. That is the Hellmann-Feynman argument, and taking it one order up is the  $2n+1$  theorem, which is why a *third* derivative needs only first-order wavefunctions.

# Contents

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| <b>1</b>  | <b>Input and invocation</b>                         | <b>3</b>  |
| <b>2</b>  | <b>Ground state</b>                                 | <b>5</b>  |
| <b>3</b>  | <b>Bands, densities of states, projections</b>      | <b>5</b>  |
| <b>4</b>  | <b>Pseudopotentials and functionals</b>             | <b>6</b>  |
| <b>5</b>  | <b>Spin, magnetism and spin-orbit</b>               | <b>7</b>  |
| <b>6</b>  | <b>Forces, stress and relaxation</b>                | <b>8</b>  |
| <b>7</b>  | <b>Response: dielectric, phonons, Raman</b>         | <b>9</b>  |
| 7.1       | Spin-polarized response . . . . .                   | 12        |
| <b>8</b>  | <b>Optical spectra and excitons: TDDFT</b>          | <b>12</b> |
| <b>9</b>  | <b>Topological invariants</b>                       | <b>14</b> |
| 9.1       | The curvature as a map: the Kubo route . . . . .    | 15        |
| <b>10</b> | <b>Things with no <code>pw.x</code> counterpart</b> | <b>16</b> |
| <b>11</b> | <b>Performance: dials, memory and GPUs</b>          | <b>16</b> |
| <b>12</b> | <b>Accuracy summary</b>                             | <b>17</b> |
|           | <b>Where to go next</b>                             | <b>18</b> |

# 1 Input and invocation

pypresso reads **pw.x input files**. The same file that runs in Quantum ESPRESSO runs here, which is what makes every comparison in this document possible.

```
from pypresso import Calculator

calc = Calculator.from_file("scf.in", pseudo_dir="pseudo", conv_thr=1.0e-8)
result = calc.get_scf()

print(result.total_energy, "Ry in", result.iterations, "iterations")
```

`conv_thr` means what it means in a `pw.x` input: it is compared against QE's `dr2`, the Hartree energy of the density residual.

## One object, bound methods

A `Calculator` is a system together with its pseudopotentials, and every calculation in this document is a method on it. The pseudopotentials are read from the names the `ATOMIC_SPECIES` card gave, resolved against `pseudo_dir` — which `from_file` defaults to the input file's own directory — so a script names an input and then names physics:

```
calc = Calculator.from_file("scf.in", pseudo_dir="pseudo")

bands = calc.get_bands(kpoints=path)      # runs the SCF first if none is cached
dos = calc.get_dos(grid=(8, 8, 8))
forces = calc.get_forces()
epsilon = calc.get_dielectric_tensor()
phonons = calc.get_phonons()

bands.plot()                             # and each result draws itself
```

Three properties are worth knowing before relying on it.

**The ground state is cached, and computed on demand.** A method that needs one and finds no cache runs an SCF, printing a line to `stderr` saying so; `get_scf()` is the explicit path and is the same code. The cache is a *single slot* holding the result together with the options that produced it — calling `get_scf` again with different options reruns and replaces it — because what it holds is the wavefunctions, and a cache keyed by option sets would quietly hold several sets of them. `calc.scf_result` reads the slot without triggering anything, and a state that did not converge is refused rather than differentiated.

**Nothing mutates.** `with_positions`, `with_cell` and `with_spin` return a *new* calculator with an empty cache; the converged state crosses as a starting guess where that is sound, which for `with_spin` is the spin-regime continuation described under *Spin, magnetism and spin-orbit* and converges in one iteration rather than twenty-five. A cached result under a moved atom would otherwise answer for the geometry it was converged at.

**The refusals pass through untouched.** Every restriction in this document — PAW Born charges, a metal's Raman tensor, a spiral with spin-orbit coupling — raises through the bound method exactly as through the function beneath it.

The functional entry points the rest of this document names — `run_scf`, `run_bands`, `dielectric_tensor` and the others — are unchanged and remain the way to drive a calculation whose state

is being managed by hand:

```
from pypresso.io.pwin import read_pw_input
from pypresso.pseudo import read_upf
from pypresso.scf.driver import Calculation, run_scf
from pypresso.system import build_system

system = build_system(read_pw_input("scf.in"))
pseudos = tuple(read_upf(f"pseudo/{s.pseudo_file}") for s in system.structure.species)
result = run_scf(system, pseudos, conv_thr=1.0e-8)
```

What such a call has to carry with the density is the rest of the converged state — `becsum` for a PAW dataset, `ns` under a Hubbard  $U$ , `tau` under a meta-GGA. None of the three can be rebuilt from the density, and the entry points refuse without them rather than computing something else. The calculator passes them.

A command line covers the common workflows:

```
python3 -m pypresso.cli inspect <qe-output> # summarise what the parser reads
python3 -m pypresso.cli dos <input> # density of states
python3 -m pypresso.cli pdos <input> # projected DOS
python3 -m pypresso.cli relax <input> # structural relaxation
python3 -m pypresso.cli stress <input> # stress tensor
python3 -m pypresso.cli spiral <input> # spin-spiral scan
```

Standard `pw.x` variables — `ecutwfc`, `ecutrho`, `ibrav`, `nbnd`, occupations, `degauss` and the rest — mean what they mean there and are not re-documented here. A handful of knobs are worth knowing because they are specific to this code or to a feature below:

|                                                   |                                                           |
|---------------------------------------------------|-----------------------------------------------------------|
| <code>mbj_c</code>                                | fixes Tran-Blaha's $c$ instead of taking the cell average |
| <code>london_s6</code> , <code>london_rcut</code> | Grimme D2's scaling and real-space cutoff                 |
| <code>cell_dofree</code>                          | which cell degrees of freedom a vc-relax may move         |
| <code>starting_ns_eigenvalue</code>               | seeds the DFT+U occupation matrix                         |
| <code>spiral_q</code>                             | the spin-spiral wavevector, in lattice coordinates        |
| <code>LOCAL_MAGNETIC_FIELDS</code>                | a card with no <code>pw.x</code> counterpart              |

**Units are Rydberg atomic units throughout** (Ry, bohr), matching QE. The only layer that speaks anything else is `io/`: Hubbard  $U$  in the `HUBBARD` card is in eV and is converted at the boundary, as `pw.x` does it.

**Refuses.** `K_POINTS gamma` selects the half-sphere storage of the gamma-point trick, which is generated but not consumed. Such a run is substituted by an explicit  $k = 0$  on the full sphere — the same physics at twice the storage — and it says so.

## 2 Ground state

The Kohn-Sham problem, generalised because ultrasoft and PAW make the overlap non-trivial:

$$\begin{aligned}\hat{H}|\psi_{n\mathbf{k}}\rangle &= \epsilon_{n\mathbf{k}} \hat{S}|\psi_{n\mathbf{k}}\rangle, & \hat{S} &= 1 + \sum_{I,ij} q_{ij} |\beta_i^I\rangle\langle\beta_j^I| \\ \hat{H} &= -\nabla^2 + v_{\text{loc}}(\mathbf{r}) + v_{\text{H}}[n] + v_{xc}[n] & & + \sum_{I,ij} D_{ij}^I |\beta_i^I\rangle\langle\beta_j^I|\end{aligned}$$

with  $n(\mathbf{r}) = \sum_{n\mathbf{k}} w_{\mathbf{k}} f_{n\mathbf{k}} (|\psi_{n\mathbf{k}}|^2 + \sum_{ij} Q_{ij}^I \langle\psi|\beta_i\rangle\langle\beta_j|\psi\rangle)$ , the augmentation term being what ultrasoft adds.  $D_{ij}^I$  is rebuilt from the potential every iteration, which is why the SCF is a fixed point in  $(n, D)$  and not in  $n$  alone. Convergence is measured as QE measures it — the Hartree energy of the residual:

$$dr^2 = \iint \frac{\delta n(\mathbf{r}) \delta n(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} d\mathbf{r} d\mathbf{r}' < \text{conv\_thr}$$

```
result = run_scf(system, pseudos, conv_thr=1.0e-10, mixing_mode="anderson")
print(result.total_energy, result.energy_terms) # QE's own term-by-term split
print(result.fermi_energy, result.homo, result.lumo)
```

**Metals** work with every smearing QE has (gaussian, marzari-vanderbilt, methfessel-paxton, fermi-dirac) and with the tetrahedron family — both linear and Blochl-corrected — usable as an occupation scheme *inside* the SCF, not only for a density of states.

**Continuation across a change of spin regime.** Give `run_scf` a previous result as `starting_from` — `System.with_spin` underneath — and it promotes a converged state into a target's variables: non-magnetic into collinear, collinear into noncollinear, spin-orbit switched on. It reaches the *same* solution as a fresh run ( $\leq 4\text{e-}8$  Ry on six cases) in as little as one iteration. The magnetization is **seeded** from the target's `starting_magnetization` when the source has none, because nothing in the SCF breaks spin symmetry on its own and an unseeded promotion would converge back to the unpolarized solution and report success.

**Convergence.** `mixing_mode` selects `anderson` (default), `linear`, `broyden`, or Kerker/Thomas-Fermi preconditioning (TF); `scf_solver` offers a Newton-Krylov alternative that is slower but reaches solutions the mixer cannot hold. `max_iterations` defaults to 100 — **a hard SCF may need more**, and hitting the cap is reported as not converged rather than as an answer.

## 3 Bands, densities of states, projections

A band structure is one diagonalisation per  $\mathbf{k}$  on a fixed density — `run_bands`, or `run_nscf` for a non-self-consistent run on a grid rather than a path. A density of states is

$$g(E) = \sum_{n\mathbf{k}} w_{\mathbf{k}} \delta(E - \epsilon_{n\mathbf{k}})$$

with  $\delta$  replaced by a smearing function or evaluated by tetrahedron interpolation. The projected DOS weights each term by  $|\langle\phi_\mu|\hat{S}|\psi_{n\mathbf{k}}\rangle|^2$ , and the spilling parameter is what is *not* captured,  $\sum_{n\mathbf{k}} w_{\mathbf{k}} f_{n\mathbf{k}} (1 - \sum_\mu |\langle\phi_\mu|\hat{S}|\psi_{n\mathbf{k}}\rangle|^2)$ .

```

from pypresso.workflows.bands import run_bands
from pypresso.workflows.dos import run_dos
from pypresso.workflows.pdos import run_pdos

bands = run_bands(system, pseudos, scf.density, kpoints=path)  # a k-path

dos, _ = run_dos(system, pseudos, scf.density, grid=(12, 12, 12),
                 scheme="tetrahedra_opt")  # -> (DensityOfStates, NSCFResult)

pdos, _ = run_pdos(system, pseudos, scf)  # -> (ProjectedDOS, NSCFResult)
print(pdos.channels[0])  # resolved by atom, l and m
print(pdos.charges.charges, pdos.charges.spilling)

```

The DOS schemes live behind a name registry (DOS\_SCHEMES), and the projected DOS feeds *the same* registry as a per-band weight, so a PDOS and a DOS are the same integration. Projections are  $\langle \phi | \hat{S} | \psi \rangle$  on Löwdin-orthogonalised pseudo-atomic orbitals. Validated against a purpose-built projwfc.x on seven cases: **6.9e-4** on a projection, **4.7e-5** on a Löwdin charge, and the spilling parameter to all four decimals it prints.

## 4 Pseudopotentials and functionals

**Kinds:** norm-conserving, **ultrasoft** and **PAW** — the two-grid split, the augmentation charge, the overlap operator, self-consistent  $D_{ij}$ , and PAW’s one-centre terms including its radial Poisson solve and spherical quadrature. UPF v2 is read.

**Functionals:** LDA (PZ), **PBE**, **revPBE**, **PBEsol**, on the plane-wave grid and on the PAW spheres. The functional comes from the pseudopotential headers unless input\_dft overrides it. Only the *energy* is written down;  $v_{xc}$ , and a GGA’s  $v_1$  and  $v_2$ , come from jax.grad of it.

**Meta-GGA, potential-only branch:** input\_dft = 'tb09' (Tran-Blaha) and 'bj06' (Becke-Johnson). These invert the rule above — they *are* potentials, there is no energy — so the consequences are enforced rather than documented: run\_scf warns that its total is not the value of any functional it minimised, and forces, stress, phonons and response all refuse. Silicon’s gap goes from LDA’s 0.49 eV to **1.13 eV** against an experimental 1.17.

**Van der Waals:** Grimme **D2** (vdw\_corr = 'grimme-d2'). Bilayer graphene matches pw.x to **3.1e-9 Ry** and binds at 3.23 Å where PBE alone has no minimum.

**Refuses.** UPF v1; an unimplemented functional (rather than silently substituting LDA); energy-carrying meta-GGAs (TPSS, SCAN, M06L), whose potential has a  $\partial E / \partial \tau$  piece acting on the wavefunction that is not written; plain *ultrasoft* under a meta-GGA, which has no partial waves to reconstruct  $\tau$  from inside the sphere where PAW does; EXX; and **D3, Tkatchenko-Scheffler, MBD and XDM** — where pw.x warns and silently runs with no correction at all.

## 5 Spin, magnetism and spin-orbit

| regime       | how to ask          | note                                            |
|--------------|---------------------|-------------------------------------------------|
| unpolarised  | default             |                                                 |
| collinear    | nspin = 2           | one Fermi level, or two under tot_magnetization |
| noncollinear | noncolin = .true.   | magnetization is a vector                       |
| spin-orbit   | + lspinorb = .true. | <i>j</i> -resolved projectors; NC, US and PAW   |

**Three spin numbers are kept apart**, and `System` exposes all three: `nspin` says which regime is in force, `npol` is the number of spinor components of a *wavefunction*, and `nspin_mag` the number of components of a *density*. They coincide at 1 and 2 and come apart at 4, where `npol` = 2 but `nspin_mag` is **one** for a nonmagnetic spin-orbit run — which is why such a run costs about what a doubled unpolarized one costs.

**Fixed occupations under `nspin` = 2** fill each channel to its own count rather than searching for a level: `tot_magnetization` gives `nelup` and `nelow`, and `iweights_only` fills `NINT( $n_{\uparrow}$ )` bands in one channel and `NINT( $n_{\downarrow}$ )` in the other. That is the *only* shape the combination has — `pw.x` refuses fixed-occupation LSDA without a `tot_magnetization`, and requires an integer one — and this package makes the same two refusals at input, before anything is allocated. Each channel’s highest occupied level is reported as `fermi_energy_up` / `fermi_energy_down` beside the overall `homo` and `lumo`, which is what `iweights` returns. Validated on an oxygen atom at `tot_magnetization` = 2, whose four-up / two-down filling is the Hund’s-rule ground state: **4.9e-9 Ry** against `pw.x`.

**Magnetic fields and constraints:** a uniform field over the cell, a field inside one atom’s sphere (through a `LOCAL_MAGNETIC_FIELDS` card that `pw.x` has no counterpart for), Elk’s `reducebf`, and all four of QE’s `constrained_magnetization` schemes. **The field’s energy is not part of the reported total** — QE prints `etcon` and never adds it, and Elk excludes its external field’s energy by the same convention.

**DFT+U:** QE’s `lda_plus_u_kind` = 0 (Dudarev) with the  $J_0/\beta$  extension, on atomic, ortho-atomic and norm-atomic projectors, read from the `HUBBARD` card. Forces come from differentiating through projectors that move with the atoms.

**Refuses.** The full (Liechtenstein) DFT+U formulation, intersite  $V$ , background channels, the orbital-resolved variant, the `wf/pseudo` projector sets, and noncollinear `ns_nc`; and PAW + GGA + a noncollinear magnetization together.

A **magnetic field in a noncollinear run with no starting moment** is refused rather than run. `domag` comes from `starting_magnetization` and from nothing else (`setup.f90`), so such a run has `nspin_mag` = 1 and a density with no magnetization channels for the field to act on. Set `starting_magnetization` for one species — any nonzero value will do, the field decides where the moment goes. Worth knowing: `pw.x` *accepts* this input and silently drops the field, because `add_bfield` writes it into the potential’s magnetization channels and `vloc_psi_nc` applies those only IF (`domag`).

The **residual solver** cannot take a fixed occupation whose filled/empty boundary cuts a **degenerate multiplet** — which is exactly what an atom’s Hund’s-rule filling does. Which member of the multiplet the eigensolver returns is arbitrary, so the density map is not a function of the density and a Newton step has nothing to converge on; it is diagnosed by name rather than left as a NaN. The mixer damps its way past it, and a *gapped* fixed-occupation LSDA cell agrees between the two solvers to 5e-12 Ry.

## 6 Forces, stress and relaxation

$$\mathbf{F}_I = -\frac{\partial E}{\partial \tau_I} \Big|_{\psi \text{ fixed}}, \quad \sigma_{ij} = -\frac{1}{\Omega} \frac{\partial E}{\partial \varepsilon_{ij}} \Big|_{\psi \text{ fixed}}$$

Both are one gradient. With ultrasoft the orthonormality constraint  $\langle \psi_n | \hat{S} | \psi_m \rangle = \delta_{nm}$  is carried explicitly, so its multiplier term — QE’s Pulay contribution — falls out of the same derivative rather than being added afterwards. A variable-cell relaxation minimises the **enthalpy**, which is why a relaxed crystal carries the applied pressure rather than having zero stress:

$$H = E + P \Omega, \quad \frac{\partial H}{\partial h} = \Omega (P \mathbf{1} - \sigma) h^{-T}$$

```
from pypresso.forces import compute_forces
from pypresso.stress import compute_stress
from pypresso.workflows.relax import run_relax
from pypresso.workflows.vc_relax import run_vc_relax

forces = compute_forces(calculation, result) # method=... for QE's own terms
print(forces.forces) # (nat, 3) in Ry/bohr

stress = compute_stress(calculation, result, terms=True)
print(stress.pressure) # kbar

relaxed = run_relax(system, pseudos, forc_conv_thr=1.0e-4)
cell = run_vc_relax(system, pseudos, press=500.0) # kbar
print(cell.pulay_error) # the frozen-basis error, reported not hidden
```

QE’s `force_lc`, `force_cc`, `force_ew`, `force_us`, `addusforce` and `force_corr` are transcribed too, behind the same registry — **as a cross-check, not as the implementation**. The two share no machinery, which is what makes disagreement informative; it is how the augmentation force’s sign and a missing gradient correction were found.

For a noncollinear or spin-orbit run the same expression is evaluated on two-component spinors: the nonlocal term takes the pseudopotential’s bare `dvan_so` (a complex  $2 \times 2$  matrix in spin space) and the orthonormality constraint takes `qq_so`, and the state is one coefficient vector of length  $2n_{\text{pw}}$  rather than two. Nothing else in the functional changes.

```
from pypresso import Calculator

calculator = Calculator.from_file("pt2-soc-force.in") # noncolin, lspinorb
forces = calculator.get_forces() # (nat, 3) Ry/bohr
stress = calculator.get_stress() # (3, 3) Ry/bohr^3
print(forces.max_force, stress.pressure_kbar)
```

Validated against `pw.x` with `tpnfor/tstress` on a four-atom noncollinear hydrogen chain (norm-conserving) to  $8.9 \times 10^{-7}$  Ry/bohr, doubled fcc platinum with spin-orbit coupling to  $7.5 \times 10^{-6}$  (ultrasoft) and  $7.3 \times 10^{-7}$  (PAW), and the stress on those three plus QE’s own three `pw_spinorbit` cases to  $\leq 1.2 \times 10^{-6}$  Ry/bohr<sup>3</sup>; and, without any Fortran at all, against a central finite difference of the frozen energy to  $6.2 \times 10^{-9}$  Ry/bohr.

A noncollinear or spin-orbit run has forces, a stress and a relaxation, but only through the *autodiff* method: `method='analytic'` is refused by name, because `force_us/stres_knl`



are transcriptions of QE's hand-derived expressions and have no spinor form. The force on an atom of a **spin spiral** is refused separately — its two spinor components live on different plane-wave spheres, so the nonlocal term needs the projectors of both;  $dE/dq$  is what a spiral has instead. **Elastic constants and electrostriction** stay refused for `noncolin`: they differentiate the same functional a third time and their first-order wavefunctions come from a Sternheimer solve that has no spinor form.

## 7 Response: dielectric, phonons, Raman

The **velocity operator** comes first and everything else uses it: it is one jvp of  $\hat{H}(\mathbf{k})$  at a frozen sphere, because  $\partial\hat{H}/\partial k_\alpha = i[\hat{H}, r_\alpha]$  in the periodic gauge. `band_velocities` exposes it — `Calculator.get_band_velocities` is the short way — and because the overlap carries a velocity too, a band velocity is  $\langle\psi|\partial_k\hat{H} - \epsilon\partial_k\hat{S}|\psi\rangle$ .

```
calc = Calculator.from_file("scf.in", pseudo_dir="pseudo")

velocities = calc.get_band_velocities()          # .velocities is (nk, nbnd, 3)
elsewhere = calc.get_band_velocities(kpoints=path)
```

Every first-order quantity then comes from the Sternheimer equation — the response of an occupied orbital to a perturbation, projected outside the occupied manifold so that no empty states and no energy denominators appear:

$$(\hat{H} - \epsilon_n\hat{S} + \alpha\hat{Q})|\Delta\psi_n\rangle = -\hat{P}_c^+ \Delta V|\psi_n\rangle$$

It is solved self-consistently, because  $\Delta V$  contains the response of the potential to the response of the density,

$$\Delta V_{\text{scf}} = \Delta V_{\text{ext}} + \int K(\mathbf{r}, \mathbf{r}') \Delta n(\mathbf{r}') d\mathbf{r}'$$

and the kernel  $K$  is one jvp of `v_of_rho` rather than a second implementation. From its solutions:

$$\epsilon_{\alpha\beta}^\infty = \delta_{\alpha\beta} + \frac{8\pi}{\Omega} \sum_{\mathbf{k}} w_{\mathbf{k}} \langle \Delta_\alpha \psi_n | \partial_\beta \psi_n \rangle, \quad Z_{I,\alpha\beta}^* = \frac{\partial F_{I\alpha}}{\partial \mathcal{E}_\beta}$$

$$C_{I\alpha,J\beta} = \frac{\partial^2 E}{\partial \tau_{I\alpha} \partial \tau_{J\beta}}, \quad \omega^2 \mathbf{e} = \frac{1}{\sqrt{M_I M_J}} C \mathbf{e}, \quad \frac{\partial \epsilon_{ij}}{\partial \tau_{I\gamma}} \text{ (Raman)}$$

The Raman tensor is the second-order energy differentiated once more along a displacement, at *frozen first-order* wavefunctions — the  $2n+1$  theorem, which is why a third derivative costs one more jvp and not a new solve.

```
from pypresso.response import (dielectric_tensor, dynamical_matrix,
                               raman_tensors, vibrational_spectrum)

eps = dielectric_tensor(calculation, scf.wavefunctions, scf.eigenvalues,
                        scf.density)
print(eps.epsilon)          # 3x3, cartesian

ph = dynamical_matrix(calculation, scf.wavefunctions, scf.eigenvalues,
```

```

scf.density, scf.becsum)
print(ph.frequencies)          # cm-1

raman = raman_tensors(calculation, scf, born_charges=True, keep_internals=True)
spectrum = vibrational_spectrum(calculation, scf) # per-mode Raman and IR
print(spectrum.raman_activity, spectrum.depolarisation)

```

born\_effective\_charges gives  $Z^*$  on its own, and mode\_activities is the bare assembly — a transcription of dynmat.x's RamanIR, kept a pure function of arrays so it is testable without a self-consistent run.

Ultrasoft and PAW work for the dielectric constant ( $\leq 1.2\text{e-}4$  against ph.x on four cases) *and for the dynamical matrix* — silicon's optical mode comes out at **513.295** and **513.378**  $\text{cm}^{-1}$  against ph.x's 513.275 and 513.404, which is tighter than the norm-conserving case's 0.05. Four things make the difference and all four switch themselves off when  $S$  is the identity: the source term is  $(dH/du - \epsilon dS/du)|\psi\rangle$ ; the first-order state has an occupied block the Sternheimer solve does not produce; the mixed state changes at *frozen* states, because the augmentation charge and the projectors travel with their atom; and the orthonormality multipliers are variables of the functional that move — as a **matrix**, since a diagonal one is not invariant under the occupied-manifold rotation the state tangent is free in. Metals work for the response and for a norm-conserving dynamical matrix; and a symmetry-reduced wedge works as of the rank- $n$  symmetriser.

**The Raman tensor is ultrasoft and PAW as well**, at **1.2e-4** on both against a central difference of  $\epsilon$  over re-converged displaced cells, where the norm-conserving control on the same script is  $6.8\text{e-}4$ . It took **two tangents that are only right together**, and either alone is worse than neither: the state tangent is  $P_c d\psi + \psi^{\text{ort}}$ , the occupied block again; and  $|b\rangle = P_c \mathbf{r}|\psi\rangle$  is *not* the solution of its own linear equation once adddvpsi\_us has applied  $S$  to it and added the augmentation dipole, so  $db$  is the tangent of a *composition* rather than of a solve. What located them is that  $\partial\epsilon/\partial\tau$  is a sum of five partial derivatives and each one can be finite-differenced on its own against a re-converged run: three agreed to  $7\text{e-}4$  and two did not.

**A trap worth knowing:** a response on a reduced  $k$ -set is a *polar vector field* and must be symmetrised as one. Running the whole grid instead is sound only if the grid is closed under the point group — a **shifted** Monkhorst-Pack grid is not.

**Refuses.** Born charges for PAW (int3\_paw against becsumort is missing, and without it the expression is wrong in sign as well as size);  $\chi^{(2)}$  and the electro-optic tensor (the second-order source term has nothing to build the  $\langle u_i | r_k | u_j \rangle$  piece from — and it is **42%** of the answer, measured); the non-analytic LO-TO term; phonons at  $q \neq 0$ ; the dynamical matrix of an ultrasoft or PAW *metal* (the wg/wk weight split was derived for a response whose becsum dependence is entirely inside dpsi, and an insulator cannot say which weight the three further tangents belong with); any response under a potential-only meta-GGA; and a shifted grid with a reduced  $k$ -set.

## Elastic constants, electrostriction and the elasto-optic tensor

The same machinery in the *strain* coordinate rather than the atomic one. **The strain response itself works on all three pseudopotential kinds** (strain\_response): the augmentation charge  $Q_{ij}(\mathbf{r})$  is a function of the *cell*, so a homogeneous strain deforms the table it is tabulated on where a displacement only translates it, and on top of that come the four terms the dynamical matrix needed. Against a central difference of *re-converged*

strained runs — which needs no `ph.x`, since `ph.x` has no strain perturbation — it agrees to **4.6e-4** (ultrasoft) and **4.7e-4** (PAW) against a norm-conserving 1.9e-4 on the same cell. The *derivatives* below are norm-conserving still — but the variational second-order energy they all stand on is not: it reproduces `dielec.f90`’s dielectric constant to **3.4e-10** (ultrasoft) and **6.9e-11** (PAW) against a norm-conserving 8.4e-10, which took four terms that each vanish when  $S$  is the identity. One of them is worth stating on its own, because it is a distinction rather than a term: a *state* is projected with  $1 - \sum |\psi\rangle\langle\psi|S$  and a *right-hand side* with  $1 - \sum S|\psi\rangle\langle\psi|$ , and collapsing the two — which is exact at  $S = 1$  — costs a factor of fourteen. The third derivative in this *strain* coordinate is what stays norm-conserving, and how far off it is has been measured rather than left unknown: both of the tangents that closed the Raman tensor transfer, and take it from **4.6e-2** to **1.3e-2** against a central difference over re-converged strained cells, where the norm-conserving control on the same script is **2.3e-4**. What is left is entirely the *db* term — the same  $-1.72$  of 112 on ultrasoft and on PAW, which is what says it is structural. One candidate for it is excluded by measurement: writing the commutator that sources *db* with the multiplier matrix rather than the frozen scalar eigenvalue closes this coordinate (1.7e-4) and breaks the *displacement* one, taking the Raman tensor from 1.2e-4 to 1.14e-3. `strain_response` solves the first-order problem for a strain — Abinit’s metric-tensor formulation, obtained for nothing here because the cell was already written in reduced coordinates — and two things follow from it:

$$C_{ijkl} = \frac{\partial \sigma_{ij}}{\partial \varepsilon_{kl}}, \quad \frac{\partial \chi_{ij}}{\partial \varepsilon_{kl}} \quad (\text{elasto-optic})$$

The second is a **third** derivative of the energy, and it comes from one `jvp` of the second-order energy at frozen first-order wavefunctions — the  $2n+1$  theorem again. Through the thermodynamic identity of Tanner, Bousquet and Janolin it gives the four electrostriction tensors  $m$ ,  $q$ ,  $M$  and  $Q$ .

```
from pypresso.response import elastic_constants, electrostriction

es = electrostriction(calculation, scf)      # solves the strain response itself
print(es.m, es.q, es.M, es.Q)              # the four electrostriction tensors
print(es.photoelastic)                     # the elasto-optic tensor
print(es.elastic.voigt)                    # C_ij in Voigt notation, GPa

# or the elastic constants alone, from a strain response you already have
C = elastic_constants(calculation, scf.wavefunctions, scf.eigenvalues,
                      scf.density, es.strain)
print(C.tensor.shape, C.voigt)
```

Converged silicon gives  $C_{11} = 198.5$  GPa and  $C_{12} = 68.9$ , against a measured 165.7 and 63.9; the three independent components of  $\partial\epsilon/\partial\varepsilon$  match a central difference over re-converged strained cells to **2e-4**, which is that difference’s own floor, and the rank-4 tensor is cubic to 3e-14 with nothing imposing it.

**Refuses.** Both are norm-conserving, `nspin = 1`, insulators and **clamped-ion**. A diverged first-order solution is refused rather than consumed in silence (`require_converged_responses`) — at QE’s `alpha_mix = 0.7` the strain response of a *slab* diverges, and consuming it gave a  $C_{ijkl}$  that was not even symmetric under  $C_{ijkl} = C_{klij}$ . The elastic constants additionally need the whole  $k$ -grid rather than a wedge, because their functional builds its own density and symmetrises it as a scalar inside the chain rule.

## 7.1 Spin-polarized response

The Sternheimer equation is solved per spin channel,

$$(H_\sigma - \varepsilon_{n\sigma}S + \alpha_{pv}Q_\sigma) |\Delta\psi_{n\sigma}\rangle = -P_{c,\sigma}^+ \Delta V_\sigma |\psi_{n\sigma}\rangle,$$

with the occupied manifold  $P_{c,\sigma}$  built from *that channel's* filling:  $N_{\uparrow}^{\text{occ}} = \text{NINT}(n_{\uparrow})$  and  $N_{\downarrow}^{\text{occ}} = \text{NINT}(n_{\downarrow})$ , which for a magnetic insulator are different numbers. Quantum ESPRESSO never meets this because LSDA doubles nks there and each spin channel is a separate  $k$ -point.

```
# chi_0 for a spin-polarized run -- the part that is validated unconditionally.
from pypresso.response.sternheimer import make_sternheimer

solver = make_sternheimer(calculation, result, spin_polarized=True)
drho = solver.chi0(dv)          # dv is (2, n1, n2, n3) on the dense grid
```

Validated against a central difference of the density under a spin-dependent probe potential —  $1.8 \times 10^{-6}$  on an antiferromagnetic hydrogen chain (a smeared metal) and  $1.1 \times 10^{-6}$  on triplet O<sub>2</sub> (the sliced branch, ultrasoft) — and against the identity that a cell with no magnetization gives the same  $\varepsilon_\infty$  at `nspin = 1` and `nspin = 2`:  $6.2 \times 10^{-14}$  on 13.806646105. The probe must differ between the channels, because  $\chi_0$  is block diagonal in spin and one equal in both would not tell the blocks apart.

**Refuses.** `tot_magnetization` together with a smearing: the two channels have separate Fermi levels and `Smearing` carries one scalar `ef`. A filling whose boundary lands inside a *degenerate multiplet* — a Hund's-rule atom is exactly this case — where the solve converges and returns an answer 100 % away from a finite difference, because which member falls below the cut is arbitrary; that is the physics, not a missing term. Born effective charges, the dynamical matrix, the strain response, the elastic constants, the Raman tensor and the electrostriction tensors for `nspin = 2`: the solve is spin-polarized, their second-derivative assembly is not. The *screened* response of a magnetic system with vacuum: for `nspin = 2` the kernel `dv_of_drho` is the second derivative of the LSDA energy in the two channel densities and diverges wherever a channel density reaches zero — measured on triplet O<sub>2</sub>, 1504 of 91125 grid points and exactly 1504 NaN. And any potential-only meta-GGA (tb09, bj06), which has no total energy to differentiate — refused by name now rather than surfacing as `v_of_rho` asking for `tau`.

## 8 Optical spectra and excitons: TDDFT

The response above is a *static* one, solved orbital by orbital. An absorption spectrum needs the frequency axis and needs the susceptibility as a **matrix** in reciprocal lattice vectors, so this is the one place where a sum over states earns its keep. `run_absorption` builds

$$\chi_{\mathbf{G}\mathbf{G}'}^0(\omega) = \frac{1}{\Omega} \sum_{\mathbf{k}} \sum_{ij} \frac{(f_i - f_j) \rho_{\mathbf{G}}^{ij} \rho_{\mathbf{G}'}^{ij*}}{\varepsilon_i - \varepsilon_j + \omega + i\eta}, \quad \rho_{\mathbf{G}}^{ij} = \langle u_i | e^{-i\mathbf{G}\cdot\mathbf{r}} | u_j \rangle$$

and solves the Dyson equation with an exchange-correlation kernel:

$$\epsilon^{-1} = 1 + X(1 - X - \tilde{f}_{\text{xc}}X)^{-1}, \quad X = v^{1/2}\chi^0v^{1/2}, \quad \tilde{f}_{\text{xc}} = v^{-1/2}f_{\text{xc}}v^{-1/2}$$

The **bootstrap** kernel of Sharma, Dewhurst, Sanna and Gross (*PRL* **107**, 186401 (2011); Elk's `fxctype = 210`) is that equation's other half, so the two are solved together to self-consistency:

$$f_{xc}^{BS} = -\frac{\epsilon^{-1}(\mathbf{q}, 0) v(\mathbf{q})}{\epsilon_0(\mathbf{q}, 0) - 1}, \quad \epsilon_0 = 1 - v\chi^0$$

It is parameter-free and diverges as  $1/q^2$ , which is what binds an electron-hole pair; ALDA's kernel is finite at  $\mathbf{q} = 0$  where  $v$  diverges, so its head and wings vanish *identically* and no amount of it can bind one.

**Quantum ESPRESSO has no counterpart.** TDDFPT/ is a Liouville-Lanczos solver with RPA and ALDA; it has no bootstrap kernel and no Dyson equation in  $\mathbf{G}$  space. The reference is Elk.

```
from pypresso.workflows import run_absorption
import numpy as np

omega = np.arange(0.0, 0.6, 0.005)          # Ry
spectrum = run_absorption(system, pseudos, scf.density, omega,
                          kernel="bootstrap", # or rpa / alda / lrc / bootstrap-1
                          nbnd=60, ecut_response=8.0,
                          broadening=0.012, scissor=0.05)

print(spectrum.absorption)                  # Im eps_M, isotropic average
print(spectrum.frequencies_ev)              # the same grid in eV
print(spectrum.alpha, spectrum.iterations)   # the LRC-equivalent head, and the
                                              # bootstrap's fixed-point count
print(spectrum.static_residual)              # how far the band truncation reaches
```

`chi_0` is the whole cost and the kernel is nearly free, so comparing kernels means building it once: `independent_response` returns it and `solve_dyson` screens it, which is what `run_absorption` does internally and what the notebook shows.

Three knobs here are this code's own. `ecut_response` (Elk's `gmaxrf`) selects the  $\mathbf{G}$ -set the matrix is built over and has its own convergence: too small and the local-field effect is missing, and the cost is quadratic in it. `scissor` is *part of the method* rather than a convenience — the paper's Eq. (3) replaces  $\chi^0$  by a gap-corrected model response, and the velocity matrix elements are renormalised by  $e_{ij}/(e_{ij} \mp \Delta)$  along with it. And `static_residual` is the diagnostic for the one error this phase cannot refuse: how many empty states the sum ran over. It is  $\epsilon_M(0)$  from this run, in RPA, minus the same quantity from the Sternheimer solve, which is band-complete; it is kernel-matched on purpose, since differencing a bootstrap number against a Sternheimer one that contains  $f_{xc}$  would measure the kernel instead.

**A trap that is invisible in the answer:** the macroscopic dielectric function is the inverse of the  $3 \times 3$  *head* of  $\epsilon^{-1}$ , not the head of the inverse of the whole matrix. The second is the microscopic  $\epsilon$ , whose head in RPA is exactly the no-local-field result; it is smooth, positive, has the right peaks and is 9% too large on silicon. Both are returned (`epsilon` and `epsilon_no_local_fields`), because the gap between them is the local-field effect.

Validated by an identity rather than against another code's spectrum: the same  $\epsilon_M(0)$  is reached by this sum over states plus a Dyson inversion and by the projected conjugate-gradient solve of `dielectric_tensor`, which shares no machinery with it and never sees an empty state. On silicon the two agree to **1.3e-2** on a constant of 22 — and that residue is the band truncation, which is why it is reported rather than tuned away. The screening = "hartree" switch on `dielectric_tensor` exists for this comparison: the identity holds only

when both sides use the same kernel.

**Refuses.** Finite  $\mathbf{q}$  (the perturbed states would live at  $\mathbf{k} + \mathbf{q}$ ); ultrasoft and PAW (every matrix element gains the augmentation charge  $Q_{ij}(\mathbf{G})$ ); metals (the  $f_i - f_j$  weight kills the  $i = j$  term, so the intraband Drude response is absent entirely — Elk’s `tdftlr` does not add it either); `nspin`  $\neq 1$ , noncollinear magnetism, spin-orbit coupling, spin spirals and a Hubbard  $U$ ; a symmetry-reduced  $k$ -set, because symmetrising  $\chi_{\mathbf{G}\mathbf{G}'}^0$  rotates *two*  $\mathbf{G}$  indices at once and the rank- $n$  symmetriser is Cartesian; the `alda` kernel with a gradient-corrected functional, since ALDA’s kernel is pointwise in the density and a GGA’s is not; and **non-convergence of the bootstrap fixed point is an error**, as it is in `tdftlr.f90`, because a spectrum from a kernel still in motion is the spectrum of nothing.

## 9 Topological invariants

Everything is built from one primitive — the overlap of occupied manifolds at neighbouring  $k$ -points,

$$M_{mn}^{(\mathbf{b})} = \langle u_{m\mathbf{k}} | \hat{S} | u_{n,\mathbf{k}+\mathbf{b}} \rangle$$

because a determinant of overlaps is blind to the unitary mixing a degenerate eigensolver leaves. The Berry curvature is the phase of a plaquette of them and the Chern number their lattice sum, which is an **exact integer on any mesh**:

$$F_{12}(\mathbf{k}) = \ln [U_1(\mathbf{k})U_2(\mathbf{k}+\hat{1})U_1(\mathbf{k}+\hat{2})^{-1}U_2(\mathbf{k})^{-1}], \quad C = \frac{1}{2\pi i} \sum_{\mathbf{k}} F_{12}(\mathbf{k})$$

$Z_2$  has two independent routes — the flow of Wannier charge centres, and, when there is an inversion centre, the parity product over the eight TRIM points:

$$(-1)^\nu = \prod_{i=1}^8 \prod_{n \text{ occ}}^{\text{Kramers}} \xi_{2n}(\Gamma_i)$$

```
from pypresso.workflows.topology import (run_z2, run_z2_3d,
                                         run_berry_curvature)

z2 = run_z2(system, pseudos, scf.density, axis=2) # Wilson loop by default
print(z2.z2, z2.gap_step) # read gap_step before believing the integer

chern = run_berry_curvature(system, pseudos, scf.density, shape=(12, 12))
print(chern) # an exact integer on any mesh

nu = run_z2_3d(system, pseudos, scf.density) # the four 3D indices
print(nu) # (nu0; nu1 nu2 nu3)
```

**Refuses.** A collinear spin-polarized run (two Hamiltonians, so two independent sets of invariants and no statement of which is meant); a  $Z_2$  without spin-orbit coupling, where every invariant is zero for a reason that has nothing to do with the band structure; the parity route without an inversion centre; and a manifold that is not separated from the band above it (`gap_tol`). A PAW run needs the converged `becsum` passed beside the density — `run_z2(..., becsum = scf.becsum)`, which `Calculator` does for you — because `ddd_paw` is a property of the wavefunctions and cannot be rebuilt from a density; leav-



ing it out is worth 1.03 eV in the eigenvalues, and an invariant computed from those is still an integer.

**Running both  $Z_2$  routes is the check.** Where they disagree the parity one is the answer, because it has no mesh and the Wilson route does. **Two things bite in a plane-wave code and both are silent:** neighbouring  $k$ -points do not share a  $G$ -sphere, so coefficients are aligned by Miller index; and the wrap at the zone edge is a *shift* of that index, without which the Chern number comes out smooth and non-integer.

## 9.1 The curvature as a map: the Kubo route

Two constructions of the same quantity, chosen by name:

$$\Omega_n^{12}(\mathbf{k}) = -2 \operatorname{Im} \sum_{m \neq n} \frac{A_{nm}^1 A_{mn}^2}{(\varepsilon_n - \varepsilon_m)^2}, \quad A_{nm}^a = \langle \psi_n | \partial_{k_a} H - \varepsilon_n \partial_{k_a} S | \psi_m \rangle$$

with `method="kubo"`, against the lattice field strength of Fukui, Hatsugai and Suzuki with `method="fhs"` (the default). The velocity operator is one `jvp` of  $H(\mathbf{k})$  at a frozen plane-wave sphere — no dense  $H(\mathbf{k})$  is ever formed — and the tangent for a crystal direction  $d$  is the reciprocal lattice vector  $\mathbf{b}_d$ .

```
from pypresso.workflows.topology import run_berry_curvature

curvature = run_berry_curvature(
    system, pseudos, density, shape=(4, 4), nocc=4, nbnd=12, method="kubo"
)
curvature.curvature          # (4, 4), Omega(k)
curvature.curvature_by_band  # (4, 4, nocc) -- see the refusal box
curvature.truncation         # what the highest empty band was worth
```

Validated against the FHS flux, which shares no machinery with it: a centred plaquette shrunk around one  $k$ -point of zincblende AlAs converges onto the pointwise Kubo value to  $3.2 \times 10^{-3}$ , and the whole  $24 \times 24$  mesh agrees plaquette by plaquette to  $1.45 \times 10^{-4}$ , improving 65-fold over a sixfold refinement. The velocity matrix elements themselves reproduce a central finite difference of  $H(\mathbf{k})$  to  $1.8 \times 10^{-9}$ , and on centrosymmetric silicon the curvature vanishes *pointwise* to  $3.5 \times 10^{-5}$ .

**Refuses.** `method="kubo"` is for the map, never for the integer: its denominator  $(\varepsilon_n - \varepsilon_m)^{-2}$  is singular at a degeneracy and its Brillouin-zone sum is an ordinary Riemann sum, which converges to an integer without ever being one — `method="fhs"` is the default and is exact on any mesh. **Ultrasoft and PAW are refused by name:** the  $\varepsilon_n \partial_{k_a} S$  term is identically zero for a norm-conserving dataset, so no validation here can see whether its off-diagonal convention is right, and  $q_{ij}(\mathbf{b})$  is the second missing piece; `method="fhs"` carries both correctly on all three dataset kinds. The sum over empty states is truncated at `nbnd` and the truncation is *reported* rather than hidden — read `BerryCurvature.truncation`, or `truncation_abs` wherever symmetry forces the curvature to vanish, since the ratio is then noise over noise. `curvature_by_band` is gauge invariant only for a non-degenerate band; inside a degenerate multiplet only the sum over the members is defined, and `curvature` (the manifold total) always is.

## 10 Things with no `pw.x` counterpart

**Spin spirals** by the generalized Bloch theorem: a spiral of wavevector  $\mathbf{q}$  is a spinor whose two components live on *different* plane-wave spheres,

$$\psi_{\mathbf{k}}(\mathbf{r}) = \begin{pmatrix} e^{i(\mathbf{k}-\mathbf{q}/2)\cdot\mathbf{r}} u^\uparrow(\mathbf{r}) \\ e^{i(\mathbf{k}+\mathbf{q}/2)\cdot\mathbf{r}} u^\downarrow(\mathbf{r}) \end{pmatrix}$$

so in the rotated frame the density is lattice periodic and the SCF, the functional and the mixer are untouched. Only two terms carry  $\mathbf{q} = |\mathbf{k} \mp \mathbf{q}/2 + \mathbf{G}|^2$  and  $v_{\text{nl}}$  — and  $dE/d\mathbf{q}$  at frozen coefficients is `jax.grad` of them.

```
from pypresso.workflows.spiral import (run_spiral_scan, relax_spiral_q,
                                       heisenberg_exchange)

scan = run_spiral_scan(system, pseudos, wavevectors) # E(q)
best = relax_spiral_q(system, pseudos) # BFGS on the reciprocal metric
print(best.wavevector, best.scf.total_energy)

J = heisenberg_exchange(scan, shells) # fit E(q) for the exchange constants
```

`heisenberg_exchange` fits  $E(\mathbf{q}) - E(0) = m^2 \sum_{\mathbf{R}} J(\mathbf{R}) [1 - \cos(\mathbf{q} \cdot \mathbf{R})]$  over a set of lattice shells, which is how a spiral scan becomes a spin model. `compute_spiral_gradient` gives  $dE/d\mathbf{q}$  on its own.

Also here: **per-atom magnetic fields**, the **topological invariants** of the previous section, and **rank- $n$  tensor symmetrisation** — `symme.f90`'s `symmatrix3/symtensor3` written at any rank.

**Refuses.** A spiral with spin-orbit coupling, permanently — it breaks the theorem, and Elk refuses it too; a spiral with symmetry, until the spin space group is written, so a spiral needs `nosym`; a spiral with ultrasoft/PAW; and spiral relaxation under a magnetic field, whose energy is outside the reported total, so the state is stationary for a different functional than the one being differentiated.

## 11 Performance: dials, memory and GPUs

**Two batching dials** control how much is in flight, and both defaults follow the platform: on a CPU, QE's loop — one  $k$ -point and one band at a time, which is what a *cache* wants — and on a GPU the whole of both axes, which is what a card wants. Either end is reachable everywhere:

```
PYPRESSO_K_BATCH=all PYPRESSO_BAND_BATCH=all python3 run.py
```

or per call, `run_scf(..., k_batch=...)`. The chunk size changes the order contributions are summed in and nothing else — round-off,  $\sim 1\text{e-}15$  Ry.

**Why the default is per platform.** A GPU has no cache to fit in and inverts the conclusion: on a ten-atom metal, `k=1,b=1` costs **4.5x** what `k=all,b=all` does on the same card, sixteen-atom silicon runs at **0.20x** — an outright loss against four CPU cores — and at thirty-two atoms `b=1` did not finish at all. So a run on a device gets both batched unless it says otherwise, and the plausible half-measure `k=all,b=1` is never the default: it is the *worst*



setting available, because it buys the batched mode’s memory with the looped mode’s kernel launches. An explicit argument beats the environment variables, which beat the platform.

**Measured GPU speedups** (one H200 against one CPU core, per SCF iteration): 2x on a cheap metal, 3-5x on norm-conserving silicon, 9-21x with an augmentation charge or a magnetization, and **83-115x on spin-orbit**. Energies agree to  $\sim 1\text{e-}9$  Ry — the same agreements the CPU already has.

**Memory.** Spin-orbit is where it bites: a 20-atom bismuth cell with a relativistic ultrasoft dataset peaks at **34.7 GB**, which fits a 141 GB card and does not fit a 32 GB one. Everything else in that sweep sits under 2 GB.

**A response costs memory according to how it is differentiated**, which is worth knowing before starting one. A *forward* response — the Sternheimer solves behind  $\epsilon$  and  $Z^*$  — costs 0.7-1.0 GB over the SCF that produced the state, and a *forward-over-reverse* one — the dynamical matrix, the Raman tensors — 1-2 GB. The *stress* is the exception and it is a *reverse* derivative through the radial transforms: **10.5 GB** on eight-atom ultrasoft silicon, ten times any of the others, and an order more than the SCF it came from. Measure your own with `tools/gpu/phase5.py <case> --property <name>`.

**Precision.** Every dtype comes from `config.Precision`; x64 is enabled before any array exists, and all validation is float64. float32 is a performance mode and **never one a correctness claim is made in**.

## 12 Accuracy summary

Against Quantum ESPRESSO on the same input:

| quantity                                  | agreement                                 |
|-------------------------------------------|-------------------------------------------|
| total energy, norm-conserving LDA         | $\sim 1\text{e-}9$ Ry                     |
| ultrasoft, PAW                            | $\leq 3\text{e-}9$ Ry                     |
| PBE / revPBE / PBEsol                     | $\leq 6\text{e-}9$ Ry                     |
| metals, every smearing and tetrahedra     | $2.5\text{e-}8$ Ry                        |
| collinear spin                            | $1.2\text{e-}9$ Ry                        |
| noncollinear magnetism                    | $2.8\text{e-}9$ Ry                        |
| spin-orbit coupling                       | $\leq 1.3\text{e-}8$ Ry                   |
| DFT+U                                     | $\leq 6.7\text{e-}9$ Ry                   |
| band structure                            | 0.0002 eV                                 |
| forces                                    | $\leq 2\text{e-}5$ Ry/bohr                |
| stress (13 cases)                         | $\leq 2.7\text{e-}7$ Ry/bohr <sup>3</sup> |
| relaxed geometry                          | $1\text{e-}6$ bohr                        |
| dielectric constant (NC, US, PAW)         | $\leq 1.2\text{e-}4$                      |
| Born effective charges (NC)               | every printed digit                       |
| phonons at $\Gamma$ , silicon             | $0.05\text{ cm}^{-1}$                     |
| phonons at $\Gamma$ , silicon (US, PAW)   | $0.02\text{--}0.03\text{ cm}^{-1}$        |
| phonons at $\Gamma$ , aluminium (metal)   | $0.002\text{ cm}^{-1}$                    |
| Raman/IR spectra vs <code>dynmat.x</code> | every printed digit                       |

**One case does not close, and is recorded rather than explained.** `bi10-soc` — ten bismuth atoms with a fully-relativistic dataset — sits  **$1.9\text{e-}4$  Ry** from QE on a total of  $-1477.737$ , which is  $1.3\text{e-}7$  relative. Both codes choose the same symmetry operations, the same  $k$ -point, the same grid and the same  $G$ -vectors, and both converge. It is in the test suite at that tolerance, not hidden.

## Where to go next

- `README.md` — the short tour and a first calculation
- `notebooks/` — 26 worked examples on concrete systems, executed and committed, each with a `.md` export beside it
- `PERFORMANCE.md` — the running performance log, CPU and GPU
- `PLAN.md` — the architecture, the phase breakdown, the trap catalogue
- `GPU.md` — the GPU roadmap